

RESEARCH ARTICLE

Optimizing RAG in Programming Education: A Comparative Study of Models and Strategies

CHRISTOPHER C. Y. YANG¹, GUAN-JIE LIU², AND CHENG-JUI YU²

¹Department of Computer Science, National Taipei University of Education, Taipei 106320, Taiwan

²Department of Computer Science and Information Engineering, National Central University, Taoyuan 32001, Taiwan

Corresponding author: Christopher C. Y. Yang (cyang@mail.ntue.edu.tw)

This work was supported by the National Science and Technology Council in Taiwan under Grant 114-2628-H-152-003, Grant 113-2410-H-152-046, Grant 112-2410-H-008-019-MY3, and Grant 111-2410-H-008-010-MY3.

ABSTRACT This study examines how Retrieval-Augmented Generation (RAG) enhances the effectiveness of generative artificial intelligence in programming education. This study compares commercial and open-source large language models within a RAG system, examining how retrieval design and prompt engineering affect response quality. By creating a database specific to machine learning courses and using the RAGAS (Retrieval-Augmented Generation Assessment) evaluation framework, a comparative analysis of five prominent models (GPT-4o, Claude-3.7-Sonnet, Gemini-2.0-Flash, Llama3.3-70b, and Ministral-8b) is conducted based on five quality metrics, namely Context Precision, Context Recall, Context Entities Recall, Faithfulness, and Response Relevance. The findings reveal considerable performance differences between different model types, with structured reasoning prompts (Chain-of-Thought and Take a Step Back) significantly improved the overall model performance. Re-ranking was identified as the most effective retrieval approach, especially in enhancing the performance of lightweight open-source models. This research provides empirical evidence for the effectiveness of economically feasible RAG systems in programming education, thus helping bridge the gap in the performance of commercial and open-source models and providing real-world solutions for resource-constrained educational settings.

INDEX TERMS Retrieval-augmented generation (RAG), prompt engineering, AI in education, programming education, LLMs, retrieval-augmented generation assessment (RAGAS), generative AI.

I. INTRODUCTION

Generative AI shows diverse potential in education, particularly for providing instant problem analysis and instructional assistance in programming education [1], [2]. Nevertheless, modern Generative AI still faces three core challenges: the lack of response accuracy, limited timeliness of information, and the inability to comprehend the contextual essence of problems [3]. The above constraints can generate irrelevant or incorrect content in real-world applications, thus lowering the credibility of education support tools [1]. Meanwhile, the performance gap between commercially created and open-source generative models is another vital issue. While commercial models offer better performance at the cost of higher expenses, open-source models often demonstrate inferior performance in dealing with complex difficulties [4].

The associate editor coordinating the review of this manuscript and approving it for publication was Jolanta Mizera-Pietraszko¹.

Accordingly, ensuring response quality while managing system costs has emerged as a key factor in the educational implementation of Generative AI.

To address these concerns, Retrieval-Augmented Generation (RAG) technology enhances the accuracy and relevance of responses by augmenting large language models with external knowledge databases [3], [5]. Studies have indicated that the use of RAG technology can substantially minimize the likelihood of models producing “hallucinations,” thereby offering more credible response content [5]. In programming education, this implies that students can obtain more accurate conceptual descriptions, algorithm analyses, and debugging tips, thereby advancing learning outcomes and programming skills [6]. Despite its potential, optimizing RAG system performance for educational environments presents significant research difficulties, particularly for resource-constrained educational institutions. RAG research currently focuses primarily on general domain applications with minimal

systematic examination of the particular needs of programming education. A significant gap in research exists regarding how model selection, prompt strategy design, and retrieval mechanism optimization in combination influence generation quality in educational environments. Furthermore, RAG systems lack standardized, quantifiable evaluation frameworks, hindering the comparison and improvement of different implementation strategies [7].

This study systematically evaluates how model type, prompt engineering, and retrieval strategies impact the response quality of a RAG system in programming education. Using the RAGAS framework on a custom knowledge base derived from machine learning course materials, we provide quantitative evidence that structured reasoning prompts cost-effectively narrow the performance gap between leading commercial and open-source models. Our analysis further identifies re-ranking as the most consistently effective retrieval strategy for enhancing both retrieval and generation quality.

A. RESEARCH OBJECTIVES AND QUESTIONS

In programming education, Generative AI has demonstrated potential for supporting learning, but response quality varies across different models. Typically, commercial models provide higher accuracy and stability but at high costs; open-source models reduce costs but often lack the completeness and correctness of commercial models. This quality gap may affect learning experiences and limit the widespread application of this technology in educational contexts.

To bridge this gap, prompt engineering and retrieval strategies optimization have become key research directions. Prompt engineering, as a technique to guide model behavior, has been proven to enhance answer accuracy and relevance; retrieval design directly affects the quality of reference information obtained by the model, thereby influencing generation results.

Therefore, this study uses a RAG architecture based on machine learning course materials to systematically explore the following three research questions, attempting to narrow the gap with commercial models. The specific research questions are:

RQ1: What differences exist in response quality between open-source and commercial generative models in a retrieval-augmented generation architecture based on machine learning course materials?

RQ2: Can improving prompts through prompt engineering enhance the response quality of different generative AI models?

RQ3: Can adjusting retrieval strategies enhance the response quality of different generative AI models?

To systematically verify these influencing factors, this study will use RAGAS as the primary evaluation framework, employing multi-indicator analysis to measure the impact of different variables on model response quality. We expect this study to provide concrete empirical results for the application

of RAG technology in programming education, clarify key factors affecting generative model response quality, and provide a valuable reference for the future development of RAG in the area of education.

II. LITERATURE REVIEW

A. APPLICATIONS OF GENERATIVE AI IN PROGRAMMING EDUCATION

Generative AI has gained significant attention in computer programming education due to its ability to provide immediate responses and encourage computational thinking. However, its effectiveness remains controversial, particularly regarding its potential for generating incorrect explanations. Ahmed et al. [1] emphasized that while ChatGPT is effective when explaining basic programming principles and answering simple questions, it often struggles with complex reasoning and debugging, with the potential of introducing reasoning pitfalls and confusion.

Similarly, Yilmaz and Yilmaz [2] identified that although ChatGPT increased students' computational thinking, self-efficacy, and motivation, there was widespread concern on students often failed to recognize inaccuracies in AI-generated outputs. This tendency of relying on immediate feedback rather than going through critical examination can lead to the acceptance of inaccurately presented material, negatively impacting learning outcomes.

However, the integration of generative AI in education is not without its challenges. While tools like ChatGPT can provide instant support, they also pose risks that warrant careful consideration. Several studies have cautioned that an over-reliance on these models can hinder the development of students' critical thinking and problem-solving skills [8], [9].

Collectively, these findings suggest that realizing the potential of generative AI in programming education requires usage guidelines, quality checks, and a strengthened role for human educators to ensure learning quality.

B. RAG ARCHITECTURE AND RELATED RESEARCH

The Retrieval-Augmented Generation (RAG) framework, introduced by Lewis et al. [3], aims to address the key limitations of large language models with regard to the incorporation of real-time knowledge updates, treatment of infrequent facts, and the elimination of hallucinations. The fundamental process involves three steps: indexing, retrieval, and generation. In the indexing step, external data are indexed into a vector space, enabling querying. In the retrieval stage, relevant information is retrieved in response to user queries. In the generation stage, the generative model combines the retrieved information into responses with greater accuracy and contextual relevance.

Early RAG models combined retrieval modules, such as BERT with pre-trained language models for the purpose of strengthening factual coherence. The REALM model, for instance, leveraged a two-stage pretraining approach to embed retrieved knowledge directly into the generation

process [10]. These early models used to employ static retrieval methods, which made them less adaptable for multimodal input or evolving knowledge requirements.

As large language models based on Transformers evolved, their combination with RAG emerged as an increasing research field. It has been found that the inclusion of vectorized retrieval outputs directly into the input of the generator can help reduce information loss in long-context processing. This proves particularly useful in academic settings, where conceptual precision and integrity of knowledge are important [5].

Jacobs and Jaschke [6] implemented a RAG-based computer programming education feedback system founded on lecture transcript vectorization and GPT-4. The feedback system provided contextualized feedback with reference to the respective lecture timestamps. Experimental results indicated that students favored lecture-referenced feedback in solving difficult problems, whereas simpler problems favored non-RAG feedback due to faster response times. The present work experimentally demonstrated the quality-response time trade-off for retrieval in educational RAG deployments and suggested situational deployment based on problem difficulty.

C. RAGAS EVALUATION FRAMEWORK AND INDICATORS

While the basic architecture of RAG systems forms a strong foundation for knowledge-intensive applications, their efficacy measurement is surrounded by challenges. The Retrieval-Augmented Generation Assessment (RAGAS) is a fully automatic evaluation framework specifically designed for RAG systems; it aims at estimating the performance of the retrieval and generation modules independently of reference responses. Since RAG systems consist of both a retriever and a language model, complete evaluation should cover both information relevance and response quality. The first theoretical foundation was suggested by the RAGAS team [7], and the later publication of updates to the official framework has further evolved and enhanced its metrics.

During the retrieval phase, the framework pays attention to Context Recall, Context Precision, and Context Entities Recall, examining the system's ability for thorough and relevant information retrieval. On the generation phase, Faithfulness and Response Relevance are employed in measuring factual accuracy and semantic coherence. These same indicators were chosen for this study since they closely fit our concentration on question-answering system performance. Details regarding the calculation and implementation of these metrics are provided in the methods section.

Though RAGAS also comprises additional metrics such as Multimodal Faithfulness, Multimodal Relevance, and Noise Sensitivity, they are excluded in this research. As our experiments consist only of text without visual information and retrieval contexts are pre-filtered, multimodality and noise consideration are not applicable.

D. DEVELOPMENT OF PROMPT ENGINEERING AND ITS IMPACT ON GENERATION QUALITY

Prompt engineering is currently a leading approach to improve the generation quality of large language models. Research indicates that output accuracy depends not only on model size and training data but also on the design of the input prompt [11]. The general principle of prompt engineering is to guide the model to produce coherent, accurate, and task-relevant responses through carefully designed prompts.

Early methods, such as zero-shot and few-shot prompting laid the foundation for task adaptation. Zero-shot prompting relies on the model's pretraining knowledge to solve problems without examples, whereas few-shot prompting provides limited examples to define task structure and logic [12]. It has been found that few-shot prompts perform extremely well for format-specific output tasks.

To address challenges in complex logical reasoning, Chain-of-Thought (CoT) prompting was developed to guide models to display intermediate reasoning steps, and thus the logic and quality of responses are improved [13]. Through the process of getting models to mimic human thinking strategies, breaking down complex issues into multiple steps and reasoning in steps. This approach has demonstrated exceptional effects in mathematical reasoning, logical analysis, and common-sense reasoning tasks.

Apart from enhancing reasoning ability, recent approaches also focus on enhancing the comprehension of input. Rephrase and Respond (RaR) approach is a two-step process in which the model first rephrases the question asked by the user prior to responding. The intermediate step assists the model in clearing up ambiguities, narrowing down semantic interpretation, and bringing the output closer to the user's intention. Experiments indicate that RaR strongly reduces semantic drift in both conversation and retrieval-based tasks, thus making it valuable for application in conversational AI systems and question-answering systems [14].

Take a Step Back (TSB) encourages models to do conceptual thinking before generating specific responses. Rather than providing an instant response, the model is prompted to consider underlying knowledge or pertinent principles first and then conduct problem-solving exercises. TSB is found to be particularly effective in domains that need contextual grounding, like scientific explanation, legal argumentation, or cross-disciplinary tasks, where rapid responses tend to produce superficial or erroneous replies. Experimental results show that TSB enhances consistency and logical coherence in the generated text, particularly for abstract or difficult questions [15].

E. IMPACT OF RETRIEVAL STRATEGIES ON GENERATION QUALITY

The design of retrieval strategies in RAG systems has a direct impact on the accuracy and coherence of the generated responses [5]. While traditional methods such as BM25, allow for effective keyword-based retrieval, they

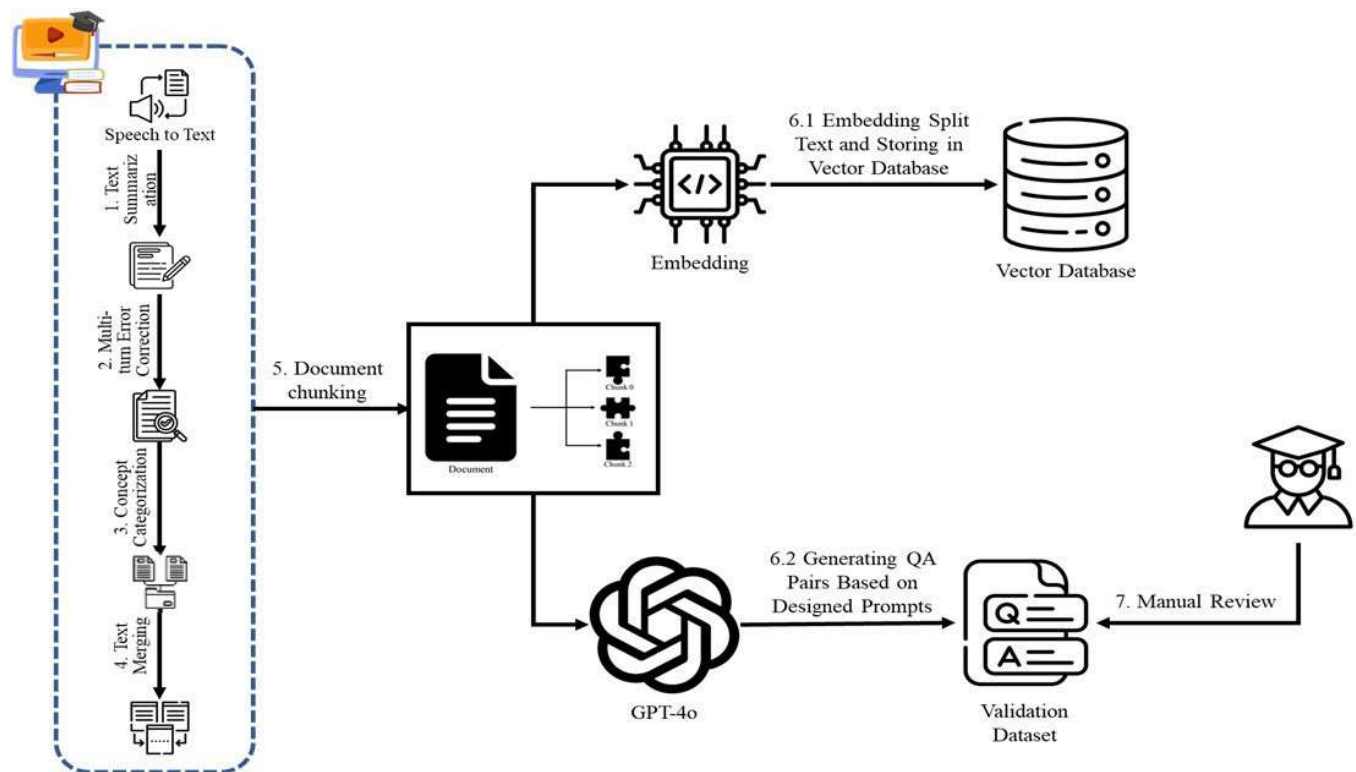


FIGURE 1. Validation dataset and vector database construction process.

often lack semantic understanding, leading to the inclusion of irrelevant or misleading information in downstream generation tasks [16].

To solve this problem, Sawarkar et al. [17] compared and evaluated various retrieval strategies, such as keyword matching, dense vector retrieval, and sparse encoder-based semantic search. From their research, they proved that a hybrid strategy, combining semantic and lexical retrieval, performed better in terms of various evaluation metrics. This effect was strongest with complex queries that demand strict term matching in addition to greater contextual relevance comprehension.

Aside from the hybrid strategy, several adaptive retrieval refinements have been suggested. Qi et al. [18] proposed RetroRAG, a framework designed to enable the model to iteratively consider the information retrieved and then revise the reasoning process. The system initially determines the relevance of the initial retrievals, then generates supporting inferences with respect to the primary entities in the query, and updates its internal context based on this evidence. Results show that this method reduces the impact of superficially pertinent but contextually irrelevant content, giving more precise answers.

Re-ranking techniques provide another refinement layer. Nogueira and Cho [19] proposed a BERT-based re-ranking approach over initial retrievals. The method employs models such as BM25 or dense retrieval to retrieve candidate documents and subsequently utilizes BERT to re-rank

them according to their semantic relevance. The approach significantly improves retrieval precision in open-domain question answering tasks by ensuring that the input finally given to the generation model is not only correct but also highly relevant.

The choice of retrieval strategy influences not only the relevance of retrieved documents but also the coherence and accuracy of the final answer. Modifying the retrieval techniques according to task complexity has become necessary for improving the overall performance of RAG systems.

III. METHOD

To explore the research questions proposed in this study, we designed a process for constructing a vector database based on machine learning course materials and establishing a validation dataset and used RAG architecture to analyze the application effectiveness of Generative AI in programming education. Through a unified experimental framework, we can systematically examine the influence of prompt design and retrieval strategies on model response quality, further clarifying the relationships between research questions.

A. VALIDATION DATASET AND VECTOR DATABASE CONSTRUCTION

In this study, to ensure that RAG technology can provide accurate and valuable content, we established a construction process for a validation dataset and used a vector database as

the core of the retrieval mechanism to match relevant content based on semantic similarity, ensuring the quality of system responses, as shown in **FIGURE 1**. Validation Dataset and Vector Database Construction Process.

Since the data used in this study mainly comes from classroom lectures, educational video resources, and related learning materials, which are mostly presented in speech or video form, we first employed speech recognition technology to transcribe speech content into text. As automatic transcription may introduce redundant information, typos, and sentence incoherence, we further used large language models to summarize and refine the transcribed text, and corrected grammatical errors and inappropriate wording through a multi-round correction mechanism to make the text closer to the original teaching content and have better readability. The validation text was created via automatic speech recognition (ASR), followed by multi-round LLM refinement and expert manual review to ensure fidelity to the original lectures. Although a formal ASR word error rate (WER) was not benchmarked in this study, we enforced a two-stage acceptance protocol: (1) LLM refinement with parameter-locked prompts to normalize technical terminology, math notation, and line breaks in code; and (2) expert verification by instructors/researchers in machine learning and programming who corrected residual issues. Representative corrections included domain term normalization (e.g., “cross-entropy,” “softmax”), symbol/notation fixes (e.g., “ $\leq/\geq$,” subscripts), and preservation of code tokens and indentation to avoid semantic drift from the original lectures. This process ensured that the final text accurately reflected the source content and was suitable for downstream retrieval and Q/A. After text preprocessing, we used large language models such as GPT-4o to classify the text content by chapters and concepts, and merged text paragraphs belonging to the same topic based on classification results, forming chapter texts with internal coherence and logical structure.

Next, the merged chapter texts were further segmented to support diverse application scenarios and to enhance the flexibility of downstream retrieval and question-answering tasks. A recursive character-level segmentation approach was adopted, targeting a chunk size of 512 tokens with a 30% overlap between consecutive segments. Because our source combines ASR transcripts with code snippets, formulas, bullet lists, and slide text, sentence boundaries are often unreliable. We therefore adopt a token-level sliding window of 512 tokens with 30% overlap. This overlap mitigates boundary loss by ensuring that content straddling a window boundary appears fully in at least one adjacent chunk, while keeping index size and latency manageable. We keep these settings fixed across all conditions to ensure comparability. Also, we acknowledge sensitivity to windowing; an ablation over 256, 512, $1024 \times 0\%$, 30%, 50% is planned for release in the public artifact, and we retain 512/30% here to keep retrieval conditions comparable.

The resulting segments were employed for both vector database construction and validation dataset generation.

For vectorization, each segment was converted into a high-dimensional embedding using OpenAI’s text-embedding-3-small model, which was selected for its strong performance in semantic similarity tasks and its capability to capture fine-grained technical concepts in programming education contexts. These embeddings allow the system to retrieve the most relevant content based on semantic similarity, thereby improving the response generation performance of the RAG system; in terms of validation dataset generation, we used large language models and designed prompts to automatically generate question-answer pairs covering various question types, simulating real learning and testing scenarios. To ensure that the generated questions are closely related to the original text topics, we further used vector embedding technology to calculate the semantic similarity between question-answer pairs and texts, automatically filtering and removing questions that do not align with learning objectives. Finally, experts in machine learning and programming manually reviewed the validation dataset to ensure its quality and practical relevance. These experts systematically evaluated each question-answer pair for accuracy and relevance to course objectives, making necessary adjustments to the automatically generated content. To detail the embedding and vector database settings, in this study, we use text-embedding-3-small (1536-d); vectors are L2-normalized and compared with cosine similarity (equivalently inner product on normalized vectors). Dense retrieval runs over an exact flat inner-product index, returning top- $k = 3$ chunks per query for all conditions to ensure fair comparison and reproducibility.

B. EVALUATION INDICATORS AND MODELS

In this research, we chose five particular indicators derived from the RAGAS framework to assess the quality of information retrieved and the quality of responses from various perspectives, which include information coverage, entity consistency, relevance, and correctness of facts. While conducting our test, we employed GPT-4o-mini as the test large language model to provide consistency and reliability of scoring among various samples. The definitions and procedural steps involved in each indicator are provided below:

- 1) **Context Recall (CR):** Context Recall calculates the number of relevant claims of a reference answer that are indeed covered by the retrieved context, emphasizing the completeness of the retrieval. This indicator ensures the preservation of key information of the reference source. It is given as:

$$CR = \frac{\text{Supported Claims}}{\text{Total Claims}} \quad (1)$$

The original reference answer is decomposed into discrete factual claims by a language model. Each of the claim is then checked within the retrieved context to determine its level of support. A claim is deemed supported if the context contains semantically

equivalent or paraphrased information that correctly reflects the claim. This way, all information required is checked. A high value of CR means a high coverage of retrieval.

- 2) **Context Precision (CP):** Context Precision at rank K is a measure that calculates the relevance of the Top- K retrieved contexts with respect to a reference answer and user input. To figure out the relevance of every individual retrieved chunk, the reference answer is directly compared using a language model. Its measurement is expressed as below:

$$CP = \frac{\sum_{k=1}^K (P@k \times v_k)}{\text{Relevant}@K} \quad (2)$$

$$P@k = \frac{TP_k}{(TP_k + FP_k)} \quad (3)$$

Here, $P@k$ is the precision at rank k , $v_k \in \{0, 1\}$ indicates relevance, and $\text{Relevant}@K$ is the number of relevant contexts within the top K results.

To score relevance, each retrieved chunk is rated in relation to the reference response by a language model. A chunk is considered relevant if it has information that is semantically close to or supports the reference response. The rating is done based on structured prompts so as to maintain consistency across samples. High CP scores are a sign of superior ranking of relevant content within the top retrieval ranks.

- 3) **Context Entities Recall (CER):** Context Entity Recall measures how well the retrieved context covers key entities that exist in the reference. It can be formulated as:

$$CER = \frac{RCE \cap RE}{RE} \quad (4)$$

where RE denotes the collection of entities present in the reference answer, while RCE represents the collection of entities identified within the retrieved contexts. Named entity recognition (NER) is utilized on both the reference text and the retrieved contexts to derive these sets of entities. This metric is particularly useful in content-based applications, where the presence of specific individuals, locations, or organizations is important. A higher CER indicates greater entity-level correspondence between retrieved context and reference text.

- 4) **Faithfulness (F):** Faithfulness assesses the factual consistency of a generated response relative to the retrieved context. It is defined as:

$$F = \frac{SC_R}{TC_R} \quad (5)$$

Here, SC_R is the number of claims in the response that are supported by the context, and TC_R represents the total number of claims in the response.

In order to compute this metric, the generated response is first broken down into individual factual claims using a language model. Then, each claim is evaluated against

the retrieved context to determine its inferability or supportability. A claim is labeled as supported if the retrieved context contains information that is semantically consistent or logically entailed. This step ensures that the response avoids including hallucinated or ungrounded information. Faithfulness scores that are higher indicate more factual alignment with supporting context.

- 5) **Response Relevance (RR):** Response Relevance measures semantic consistency between the generated answer and the original question. It is calculated as:

$$RR = \frac{1}{N} \sum_{i=1}^N \cos(E_{gi}, E_o) \quad (6)$$

where E_{gi} is the i -th reverse-generated question embedding and E_o is the embedding of the original user input. To calculate this metric, the model initially creates a set of generated questions based on the response. The questions aim to encapsulate the most important information of the response. Sentence embeddings are then generated for all the generated questions and the user query. The cosine similarity of each pair is computed and then averaged to derive the final score. High values demonstrate a greater semantic relationship and greater relevance of the answer to the user's query.

To ensure a consistent and replicable evaluation process, we employed a single large language model, GPT-4o-mini, as the judge for scoring all outputs against the RAGAS metrics. This approach, commonly referred to as “LLM-as-judge”, was chosen to automate the assessment of the 280 question-answer pairs across all experimental conditions, thereby ensuring uniform application of the evaluation criteria. To mitigate variability and enhance the reliability of the automated scoring, we implemented the following fixed parameters for the judge model:

- 1) Temperature: Set to 0.2 to minimize randomness and promote deterministic outputs.
- 2) Seed: A fixed seed was used for all evaluation runs to ensure full reproducibility of the scores.

For reproducibility, we provide the exact prompt templates for all strategies, specify the few-shot example count ($k=3$), and list token budgets in Appendix.

C. EVALUATION MODELS

In this study, we selected five representative large language models for system evaluation based on the “Chatbot Arena LLM Leaderboard” from Hugging Face. The selection criteria emphasized covering models of different scales, architectures, and application scenarios to comprehensively compare the adaptability of the RAG architecture:

- 1) **GPT-4o:** A multimodal large language model launched by OpenAI in 2024, capable of processing text, image, and voice inputs in real-time. It is a leading commercial model with high stability and performance standards.

As a current leading representative of commercial models, it possesses high accuracy and diverse capabilities. Selecting this model as a benchmark enables effective evaluation of the relative performance of other models under different conditions.

- 2) **Claude-3.7-Sonnet**: Launched by Anthropic in 2025, focusing on long-form reasoning, semantic consistency, and safety, with efficient handling of long contexts. It performs robustly in complex tasks and has notable differences from other commercial models in training principles and response style, thus demonstrating the diversity in design and output among commercial models.
- 3) **Gemini-2.0-Flash**: A high-performance language model launched by Google DeepMind, featuring low latency and fast responses, supporting multimodal inputs and long contexts. Compared to Claude-3.7-Sonnet and GPT-4o, Gemini-2.0-Flash's architecture is more inclined toward practical, fast application scenarios. Including this model helps evaluate the effect of retrieval architecture on high-efficiency models.
- 4) **Llama3.3-70b**: Meta's latest open-source large language model, with 70 billion parameters, fine-tuned for instruction-based tasks. This model is one of the most outstanding open-source models currently available, suitable for direct comparison with commercial models to evaluate the potential performance space of open-source architectures after prompt or retrieval strategy adjustments.
- 5) **Ministral-8b**: A lightweight open-source model launched by Mistral AI, featuring efficient attention mechanisms and long context processing capabilities, designed to be closer to edge computing and low-resource deployment needs. With its smaller scale but still decent generation capabilities, it serves as a representative of small to medium models, helping us observe the practical benefits of prompts and retrieval strategies for lightweight models.

D. RETRIEVAL STRATEGY DESIGN AND IMPLEMENTATION

Four retrieval strategies were used in this research to examine the effect of different retrieval strategies on the response quality of the RAG framework. Each strategy has certain characteristics in information extraction mechanisms, suitable for different application scenarios. The following are the implementation methods in detail for each strategy:

- 1) **Basic Vector Similarity Retrieval**: The simple approach employs a dense vector retrieval model, transforming queries and documents into representations in a high-dimensional vector space, thereby identifying semantically relevant documents based on vector similarity comparisons. This approach focuses on deep semantic comprehension, processing synonyms and complicated conceptual expressions effectively, which

makes it appropriate for uncovering invisible semantic relationships. The paper uses the embedding model provided by OpenAI to create vector representations and carries out similarity retrieval using a vector database. In all runs we fix cosine similarity over L2-normalized 1536-d embeddings, using an exact flat inner-product index with $\text{top-k} = 3$ to keep retrieval settings consistent across conditions.

- 2) **Hybrid Retrieval Strategy**: The hybrid retrieval strategy integrates the power of sparse and dense retrieval approaches, wherein these two approaches are effectively balanced with the help of adjustable weight parameters. Sparse representation, based on the BM25 algorithm, captures word overlap, while dense representation emphasizes semantic similarity. The hybrid strategy is particularly useful for dealing with queries involving specialized vocabulary words or infrequent words, as it takes into account both word matching and semantic comprehension together to provide more complete retrieval results. In this study, we compute a dense cosine-similarity score using `text-embedding-3-small` (1536-d) and a sparse BM25 score; both scores are min-max normalized and combined by a simple weighted sum:

$$s_{\text{hybrid}} = 0.5 s_{\text{dense}} + 0.5 s_{\text{BM25}}. \quad (7)$$

Candidates are ranked by s_{hybrid} for downstream selection. Dense similarity uses cosine distance in the vector database.

- 3) **Self-Reflective Retrieval Strategy**: Self-reflective retrieval strategy employs a feedback cycle of retrieval, evaluation, and additional retrieval. The system first performs straightforward vector retrieval to obtain some starting documents, then analyzes whether the retrieved documents sufficiently cover the question. If the information is insufficient, the system itself generates additional query keywords to assist further retrieval. This method enables continual evidence gathering and consideration, particularly suitable for complex questions, as it is able to adaptively redirect retrieval and integrate novel evidence with existing knowledge.
- 4) **Re-ranking Retrieval Strategy**: This strategy utilizes a two-step retrieval process, initially retrieving a large set of candidate documents and subsequently assessing their relevance by using a ranking model. The first stage uses base vector retrieval methods to obtain initial results, while the second stage holds these candidate documents to a high standard using a dedicated ranking model. The re-ranking process considers not just lexical overlap between queries and documents but also evaluates conceptual relevance and content quality, and thus significantly enhances retrieval precision. After initial retrieval, we apply a cross-encoder re-ranker (`BGE-reranker`) in a zero-shot setting to re-score candidates with query-document cross attention; the

final top- k passed to generation is $k = 3$ for all experiments.

E. ETHICAL APPROVAL AND DATA USE

This study used instructor-authored course videos and teaching materials as source content for transcription and retrieval; no student records, personal data, or learner interactions were collected or processed. Human experts participated only as content verifiers of non-personal instructional material; no demographic or identifying information was gathered. All materials were used under the applicable institutional/course permissions and platform terms; processed transcripts/segments were retained for research purposes without public redistribution. Access to research artifacts was restricted to the authors on secured, access-controlled storage.

IV. RESULTS

A. RQ1: COMPARISON OF RESPONSE QUALITY BETWEEN OPEN-SOURCE AND COMMERCIAL GENERATIVE MODELS

Due to deviations from normality (Shapiro-Wilk test, $p < 0.0001$) and homogeneity of variance violations (Levene's test, $p < 0.01$), non-parametric tests (Kruskal-Wallis and Mann-Whitney U) were used to evaluate differences between the models. This section compares the generation response quality of five large language models under the RAG architecture, focusing on two indicators that are directly related to generation in the RAGAS model: Response Relevance (RR) and Faithfulness (F). Each model was evaluated using a dataset of 280 question-answer pairs. All pairwise Mann-Whitney tests were adjusted using the Holm method within each family (models $\times$ metric). For key contrasts, we also report exact p-values and effect size r with 95% CIs.

While RAGAS includes five indicators, this section focuses on generation quality (Response Relevance and Faithfulness). Retrieval-focused metrics (Context Recall, Precision, and Entity Recall) are addressed in Section C.

As shown in TABLE 1, on response relevance, GPT-4o performed the best ($RR=0.7839$), reflecting its high ability for semantic comprehension, with Gemini-2.0-Flash performing the worst ($RR=0.6759$). Both Claude-3.7-Sonnet and Llama3.3-70B performed equally ($RR=0.7420$), with Ministral-8b performing slightly better ($RR=0.7466$). On faithfulness, Claude-3.7-Sonnet performed best ($F=0.9131$), closely followed by Gemini-2.0-Flash ($F=0.9118$). GPT-4o and Llama3.3-70b achieved comparatively middle-range faithfulness scores ($F=0.8693$ and $F=0.8625$ respectively), while Ministral-8b received the lowest ($F=0.8037$).

The Kruskal-Wallis test revealed significant differences between the models in response relevance ($H(4) = 45.53$, $p < 0.0001$) and faithfulness ($H(4) = 86.79$, $p < 0.0001$). Following post-hoc Mann-Whitney U tests, Gemini-2.0-Flash proved to have significantly lower performance in

TABLE 1. Model comparison on response quality (mean [95% CI]).

LLMs	RR (mean [95% CI])	F (mean [95% CI])
GPT-4o	0.7839 [0.736, 0.832]**	0.8693 [0.830, 0.909]
Claude-3.7-Sonnet	0.7420 [0.691, 0.793]	0.9131 [0.880, 0.946]**
Gemini-2.0-Flash	0.6759 [0.621, 0.731]*	0.9118 [0.879, 0.945]**
Llama3.3-70b	0.7420 [0.691, 0.793]	0.8625 [0.822, 0.903]
Ministral-8b	0.7466 [0.696, 0.798]	0.8037 [0.757, 0.850]*

Note: Entries are mean [95% CI]. **Total samples for this table:** $N = 280$ question-answer pairs per model;

** $p < 0.01$ indicates significant differences from other models;

* $p < 0.05$ indicates significant difference from at least one other model;

Significance based on Mann-Whitney U tests;

RR and F are computed following Eqs. (5)–(6).

response relevance compared to all other models (all $p < 0.05$). In faithfulness, commercial models (Claude-3.7-Sonnet and Gemini-2.0-Flash) proved to have a significant lead over open-source models ($p < 0.001$).

Following result aggregation by model type, no statistically significant difference was found between open-source and commercial models for response relevance ($U=229,923.0$, $p=0.51$, $r = 0.022$), indicating a negligible effect size and thus comparable levels of semantic alignment. Conversely, a significant difference was found for faithfulness ($U=289,478.5$, $p < 0.0001$), with a small to medium effect size ($r = 0.231$) favoring commercial models.

B. RQ2: IMPACT OF PROMPT ENGINEERING ON DIFFERENT MODELS' RESPONSE QUALITY

To verify RQ2, this section analyzes how the four prompt strategies mentioned earlier affect the response quality of each generative model. Using the original output from RQ1 as a Baseline, which only included user questions and retrieved content without any guiding prompts or demonstration examples. We evaluated how each strategy improved models' response relevance (RR) and faithfulness (F). The experimental results are shown in TABLE 2.

Results show that GPT-4o demonstrated high stability across all prompt strategies, especially with the CoT strategy, achieving the highest response relevance ($RR=0.8461$) and excellent faithfulness ($F=0.9062$). The TSB strategy also significantly enhanced its faithfulness to 0.9115, indicating that these reasoning-oriented prompt designs can effectively promote the model to produce logically clear and fact-based content.

Claude-3.7-Sonnet particularly excelled in faithfulness performance, achieving F values as high as 0.9150 and 0.8970 with Few-shot and TSB strategies respectively, and also obtained good response relevance ($RR=0.8187$) with CoT prompts, showing good adaptability to various prompt strategies.

Although Gemini-2.0-Flash had slightly lower overall response relevance than GPT-4o and Claude-3.7-Sonnet, it demonstrated the highest faithfulness ($F=0.9293$) with the TSB strategy. However, the model showed relatively unstable performance with Few-shot and RaR strategies, indicating it

might be more suited to structured reasoning processes rather than more free-form prompt styles.

Open-source models also showed distinct prompt benefits. Llama3.3-70b, as an open-source model, improved to $RR=0.8350$ and 0.8276 with CoT and TSB strategies respectively, while maintaining relatively stable faithfulness around $F\approx 0.87$. Ministral showed the most notable improvement after applying prompt strategies. With the CoT strategy, its RR increased to 0.8431 , and faithfulness grew from the baseline of 0.8037 to 0.8505 , performing similarly to large commercial models.

To validate these observations, we conducted statistical analysis using non-parametric tests due to violations of normality assumptions. The Kruskal-Wallis tests confirmed significant differences across all five models in both response relevance (H(4) values ranging from 87.51 to 176.40, all $p<0.0001$) and faithfulness (H(4) values ranging from 13.44 to 93.81, all $p<0.01$).

The post-hoc Mann-Whitney U tests further revealed that structured reasoning prompts (CoT and TSB) consistently outperformed the baseline. Notably, the effect sizes, calculated using rank-biserial correlation, were generally in the small-to-medium range. For instance, the TSB strategy, which yielded strong improvements across models, produced a medium effect on Response Relevance for Gemini-2.0-Flash ($r = 0.249$, $p < .001$) and a small effect on Faithfulness for Ministral-8b ($r = 0.186$, $p < .001$). These findings statistically confirm that prompt engineering can effectively narrow the performance gap between commercial and open-source models, even if the magnitude of the effect is more modest than previously interpreted.

C. RQ3: IMPACT OF RETRIEVAL STRATEGY IMPROVEMENTS ON MODEL RESPONSE QUALITY

To explore RQ3, this section analyzes the impact of retrieval strategies on model response quality. Considering model resource and performance differences, we selected three models from the results of RQ1 and RQ2: GPT-4o, Gemini-2.0-Flash, and Ministral-8b, to achieve a balance between resource usage and research depth.

The research considered four different retrieval methods: (1) Baseline, described as a basic vector similarity retrieval method; (2) Hybrid, defined as a hybrid approach combining both keyword and vector retrieval methods; (3) Self-Reflective, described self-reflective retrieval method which involves a self-evaluation process to refine retrieval effectiveness; and (4) Re-ranking, represented as a re-ranking process of top-ranked outputs. The assessment used five measures from the RAGAS framework, namely context precision (CP), context recall (CR), context entity recall (CER), response relevance (RR), and faithfulness (F), as presented in TABLE 3.

The experiment findings show that the Re-ranking strategy generally resulted in the most significant improvements across the models. A notable exception was the Faithfulness score for Gemini-2.0-Flash, which peaked under the Hybrid

strategy (0.9476) rather than Re-ranking (0.9352). For GPT-4o, in particular, its execution resulted in a CP increase from a baseline of 0.9512 to 0.9835 , from 0.8568 to 0.8768 for CR, and from 0.7839 to 0.8055 for RR. Moreover, the faithfulness metric also reflected an increase (0.8693 to 0.8875), indicating that re-ranking methods can effectively improve retrieval outputs as well as text generation. On the other hand, while the Hybrid approach reflected limited improvements for some metrics, the overall effect was not as notable as that of Re-ranking and Self-Reflective strategies.

The Gemini-2.0-Flash model showed similar patterns, with Re-ranking achieving the highest CP and CR values as high as 0.9846 and 0.8791 . Importantly, though Gemini itself produced relatively lower RR performance compared to other models, its response relevance saw a significant increase after the improvements in retrieval methods, from a baseline of 0.6759 to 0.7169 through Re-ranking.

In Ministral-8b, which was an open-source lightweight model, the improves provided by retrieval methods were most obvious. The adoption of Re-ranking strategy raise its CR to 0.8838 , its CER to 0.3250 , and improved its RR from a baseline of 0.7466 to 0.7873 , with gains most significant among the three models. This suggests that appropriate retrieval methods yield greater benefits for computationally-constrained models.

Further exploration of the impact of the Self-Reflective approach found that it performed positively in context precision improvement, though its ability to increase relevance to responses was less than that of the Re-ranking approach. The reason might be that its main focus in correcting itself while retrieving was working in a way that ends up reducing attention on relevance to the original query. This can contrast with the Re-ranking approach that improves retrieval quality by directly assessing query-result relevance, promoting more balanced improvements in context precision, content recall, and response relevance.

These findings were supported by statistical analysis. The Kruskal-Wallis tests showed that significant differences in retrieval strategies were found in all models and metrics (all $p<0.01$), with context precision having the greatest effects (H(3) values ranging from 84.00 to 97.59, $p<0.0001$). The Mann-Whitney U tests revealed that Re-ranking consistently outperformed the baseline in all models for context precision. However, contrary to the initial interpretation, the effect sizes were in the small-to-medium range. For context precision, the rank-biserial correlation (r) for Re-ranking compared to the baseline was 0.160 for GPT-4o, 0.176 for Gemini-2.0-Flash, and 0.143 for Ministral-8b (all $p < .01$). The lightweight Ministral-8b model received the most improvements through Re-ranking, with significant enhancements across all five metrics ($p<0.01$).

V. DISCUSSION AND LIMITATIONS

This study systematically evaluated the impact of different model types, prompt engineering strategies, and retrieval

TABLE 2. Comparative effects of prompt strategies on response quality (mean [95% CI]).

LLMs	Strategy	RR	F
GPT-4o	Baseline	0.7839 [0.736, 0.832]	0.8693 [0.830, 0.909]
	Few-shot	0.7682 [0.719, 0.818]	0.8792 [0.841, 0.917]
	CoT	0.8461 [0.804, 0.888]**	0.9062 [0.872, 0.940]*
	RaR	0.8381 [0.795, 0.881]**	0.8909 [0.854, 0.927]
	TSB	0.8289 [0.785, 0.873]**	0.9115 [0.878, 0.945]**
Claude-3.7-Sonnet	Baseline	0.7420 [0.691, 0.793]	0.9131 [0.880, 0.946]
	Few-shot	0.7585 [0.708, 0.809]	0.9150 [0.882, 0.948]
	CoT	0.8187 [0.774, 0.864]**	0.8938 [0.858, 0.930]
	RaR	0.7977 [0.751, 0.845]*	0.8856 [0.848, 0.923]
	TSB	0.8171 [0.772, 0.862]**	0.8970 [0.861, 0.933]
Gemini-2.0-Flash	Baseline	0.6759 [0.621, 0.731]	0.9118 [0.879, 0.945]
	Few-shot	0.6545 [0.599, 0.710]	0.9359 [0.907, 0.965]*
	CoT	0.8112 [0.765, 0.857]**	0.8985 [0.863, 0.934]
	RaR	0.7930 [0.746, 0.840]**	0.8698 [0.830, 0.909]
	TSB	0.8280 [0.784, 0.872]**	0.9293 [0.899, 0.959]
Llama3.3-70b	Baseline	0.7420 [0.691, 0.793]	0.8625 [0.822, 0.903]
	Few-shot	0.7732 [0.724, 0.822]	0.8670 [0.827, 0.907]
	CoT	0.8350 [0.792, 0.878]**	0.8794 [0.841, 0.918]*
	RaR	0.8256 [0.781, 0.870]**	0.8571 [0.816, 0.898]
	TSB	0.8276 [0.783, 0.872]**	0.8775 [0.839, 0.916]
Ministral-8b	Baseline	0.7466 [0.696, 0.798]	0.8037 [0.757, 0.850]
	Few-shot	0.6931 [0.639, 0.747]	0.7819 [0.734, 0.830]
	CoT	0.8431 [0.800, 0.886]**	0.8505 [0.809, 0.892]**
	RaR	0.8308 [0.787, 0.875]**	0.8457 [0.803, 0.888]**
	TSB	0.8485 [0.807, 0.890]**	0.8712 [0.832, 0.910]**

Note: Entries are *mean* [95% *CI*] with $N = 280$ items per *model* $\times$ *prompt* condition; * $p < 0.05$, ** $p < 0.01$ relative to the Baseline of the same model (Mann–Whitney U); RR and F are computed following Eqs. (5)–(6).

TABLE 3. Effect of different retrieval strategies on RAGAS evaluation metrics (mean with 95% CI).

LLMs	Strategy	CP	CR	CER	RR	F
GPT-4o	Baseline	0.9512 [0.926, 0.976]	0.8568 [0.816, 0.898]	0.3152 [0.261, 0.370]	0.7839 [0.736, 0.832]	0.8693 [0.830, 0.909]
	Hybrid	0.9574 [0.934, 0.981]	0.8629 [0.823, 0.903]	0.3082 [0.254, 0.362]	0.7769 [0.728, 0.826]	0.8795 [0.841, 0.918]
	Self-Reflective	0.9778** [0.961, 0.995]	0.8672* [0.827, 0.907]	0.3144 [0.260, 0.369]	0.7879 [0.740, 0.836]	0.8705 [0.831, 0.910]
	Re-ranking	0.9835** [0.969, 0.998]	0.8768** [0.838, 0.915]	0.3240* [0.269, 0.379]	0.8055** [0.759, 0.852]	0.8875** [0.850, 0.925]
Gemini-2.0-Flash	Baseline	0.9515 [0.926, 0.977]	0.8658 [0.826, 0.906]	0.3181 [0.264, 0.373]	0.6759 [0.621, 0.731]	0.9118 [0.879, 0.945]
	Hybrid	0.9572 [0.933, 0.981]	0.8690 [0.829, 0.909]	0.3052 [0.251, 0.359]	0.6846 [0.630, 0.739]	0.9476* [0.921, 0.974]
	Self-Reflective	0.9752** [0.957, 0.993]	0.8686 [0.829, 0.908]	0.3177 [0.263, 0.372]	0.7062* [0.653, 0.760]	0.9440* [0.917, 0.971]
	Re-ranking	0.9846** [0.970, 0.999]	0.8791** [0.841, 0.917]	0.3194 [0.265, 0.374]	0.7169** [0.664, 0.770]	0.9352* [0.906, 0.964]
Ministral-8b	Baseline	0.9539 [0.929, 0.978]	0.8547 [0.813, 0.896]	0.3118 [0.258, 0.366]	0.7466 [0.696, 0.798]	0.8037 [0.757, 0.850]
	Hybrid	0.9550 [0.931, 0.979]	0.8590 [0.818, 0.900]	0.3137 [0.259, 0.368]	0.7346 [0.683, 0.786]	0.8205 [0.776, 0.865]
	Self-Reflective	0.9725** [0.953, 0.992]	0.8679* [0.828, 0.908]	0.3207* [0.266, 0.375]	0.7556 [0.705, 0.806]	0.8355* [0.792, 0.879]
	Re-ranking	0.9835** [0.969, 0.998]	0.8838** [0.846, 0.921]	0.3250** [0.270, 0.380]	0.7873** [0.739, 0.835]	0.8264* [0.782, 0.871]

Note: Cells show *mean* on the first line and [95% *CI*] on the second line, with $N = 280$ items per *model* $\times$ *retrieval* condition; * $p < 0.05$, ** $p < 0.01$ relative to the Baseline of the same model (Mann–Whitney U); RR and F are computed following Eqs. (5)–(6).

strategies on response quality in the RAG architecture. Through in-depth analysis of experimental results, we have identified several noteworthy phenomena and potential improvement directions, while also recognizing the limitations of the research.

A. EDUCATIONAL SIGNIFICANCE OF PERFORMANCE DIFFERENCES BETWEEN COMMERCIAL AND OPEN-SOURCE MODELS

These performance differences observed between open-source and commercial models within the RAG architecture have

profound implications for their application in education and for the selection of AI technologies [1], [2]. Statistical analysis confirmed significant differences among the models in response relevance ($H(4)=45.53$, $p<0.0001$). However, when comparing model types, the difference between commercial and open-source models was not statistically significant, and the rank-biserial correlation revealed a negligible effect size ($r = 0.022$). This finding suggests that the RAG architecture can effectively mitigate performance gaps in semantic comprehension, making the practical difference between model types minimal for this metric. Conversely, for faithfulness, the effect size was small to medium ($r = 0.231$), indicating that model selection remains an important consideration for educational applications that demand high levels of factual accuracy.

Accuracy of lightweight models in response relevance is especially significant in educational settings. The Mann–Whitney U test showed that there was no statistically significant difference in the response relevance of Minstral-8b and commercial models, such as GPT-4o, even though there were significant differences in the numbers of parameters and underlying resources. This result is particularly valuable for resource-constrained educational settings where the ongoing API usage costs of commercial LLMs may be unsustainable. Those institutions with less computational equipment can potentially leverage RAG-enhanced lightweight models executed locally for satisfactory performance on significant measures of pedagogical interaction quality, particularly semantic understanding.

The findings provide empirical evidence for the possibility of using lightweight open-source models in AI-facilitated education technology. Although commercial models remain superior in fact accuracy, the agreement in semantic relevance of answers suggests that RAG-enhanced open-source models can make education content facilitated by AI more accessible, particularly in environments characterized by budgetary or infrastructural limitations.

B. PROMPT STRATEGIES FOR COST-EFFECTIVE OPTIMIZATION METHODS

Prompt strategy analysis shows that CoT and TSB strategies differ in how they enhance model performance and are suitable for specific models. The cross-model utility of CoT and TSB strategies reflects the general role of structured thinking in generation tasks [13], [15]. The fact that these strategies work for several models is probably because they have been exposed to enormous quantities of text that contain reasoning processes during pre-training, making them very receptive to step-by-step guidance. This type of design is consistent with educational scaffolding theory principles, which focus on providing students incremental and structured assistance to help internalization of complex problem-solving techniques [20]. Through decomposition of tasks into simpler sub-components, CoT and TSB optimize factual coherence while simultaneously decreasing cognitive load,

particularly in learning scenarios where excessively complex explanations would overwhelm students. Notably, this increase is proportionally higher for open-source lightweight models, which implies that effective thought guidance can be an effective means to lighten parameter scale limitations. However, few-shot strategy underperformed in some models, with this decline possibly being the product of insufficient context matching between examples and questions, causing models to excessive compliance with examples at the cost of question uniqueness [12].

For AI-supported learning systems, these results suggest developers should prioritize effective prompt strategies for specific models over simply selecting larger ones. Particularly for programming education, the TSB strategy that promotes step decomposition has a high degree of alignment with problem decomposition thinking in programming instruction, with the potential to improve response quality and develop students' computational thinking simultaneously.

C. INFLUENCE OF RETRIEVAL STRATEGIES

In RQ3, the consistent and significant advantage of the Re-ranking strategy reveals the key value of two-stage evaluation in the retrieval process [19]. This strategy is effective because it simulates the “preliminary screening-fine evaluation” dual-stage cognitive process that human experts use when answering professional questions. In contrast, the Hybrid strategy was less effective than expected, even negatively impacting response relevance for Minstral-8b [17]. This phenomenon may have several causes: First, hybrid retrieval involves weight distribution issues between different retrieval methods, which if improperly set, may lead to relative degradation of retrieval results; second, keyword retrieval and vector retrieval may have semantic conflicts, potentially increasing noise without good coordination mechanisms; third, lightweight models have limited ability to handle multi-information integration and may not effectively distinguish between high-value and low-value information in hybrid retrieval results.

Notably, the Self-Reflective strategy performed well in improving context precision but was less effective than Re-ranking in enhancing response relevance, reflecting the difference in information filtering logic between “model self-evaluation” and “direct ranking based on question relevance.” The former focuses more on self-assessment of content itself, while the latter directly ranks based on content relevance to the question; from experimental results, the latter better enhances final generation quality.

These findings offer direct guidance for designing educational RAG systems: when resources permit, implementing two-stage retrieval mechanisms should be a priority, especially for concept-intensive materials.

D. RESEARCH LIMITATIONS AND FUTURE DIRECTIONS

Although this study provides empirical evidence for RAG technology applications in programming education, several important limitations should be considered. First, a primary

limitation is the narrow scope of our dataset, which is exclusively focused on machine learning course content. While this allows for a deep analysis within one complex domain, the unique knowledge structure and conceptual relationships of machine learning may not be representative of other academic fields. Programming education emphasizes logical design and algorithmic thinking, whereas disciplines in the humanities or social sciences rely more on critical analysis and argumentation. Therefore, the effectiveness of the evaluated RAG strategies may not be directly generalizable to other subjects.

Second, the sample size of 280 question–answer pairs, while sufficient for initial comparisons, may constrain the statistical robustness of our findings, especially when data are segmented across five models and multiple prompting and retrieval strategies. A larger and more diverse dataset is needed to draw more definitive conclusions about performance variances across question types and difficulty levels. Future research should validate these findings with a more extensive dataset that broadens topic coverage to enhance generalizability. For instance, programming education emphasizes logical design and problem solving, whereas humanities fields emphasize conceptual analysis and critical thinking; such differences may lead to varying effectiveness of RAG systems across subjects, and cross-disciplinary comparisons are therefore warranted. In addition, we asked two domain experts to rate a few representative items and compared their judgments with our automatic metrics; we report three examples inline: (1) concept-definition item—experts: F=5/5, RR=5/5; automatic: F=High, RR=High. (2) multi-step reasoning item—experts: F=4/5 (minor omission), RR=5/5; automatic: F=Medium–High, RR=High. (3) off-topic distractor item—experts: F=2/5, RR=2/5; automatic: F=Low, RR=Low. These spot checks qualitatively align human judgments with our automatic scores, while a full-scale validation is left for future work.

At the methodological level, we acknowledge a potential vendor-aligned bias because the judge model (GPT-4o-mini) is from the same developer as one of the evaluated models (GPT-4o). Such alignment could favor outputs that share stylistic or structural features with the judge and, in turn, affect validity. To preserve within-study comparability, we retain a single, parameter-locked judge; as complementary evidence, two domain experts rated three representative items and their judgments qualitatively aligned with the automatic scores reported here. To mitigate judge bias in future replications, we will run a stratified $\sim 10\%$ subset (by model $\times$ prompt $\times$ retrieval) with an alternative LLM judge and report inter-judge agreement (e.g., Spearman's ρ or Cohen's κ); disagreements beyond a predefined tolerance will be adjudicated by two human raters using a brief rubric. Further, since we did not compute a formal ASR WER due to the lack of aligned, time-coded references, we document the verification protocol and representative correction types here and will release aligned samples and edit logs in a future artifact to enable precise WER reporting.

Future work should aim to replicate these experiments with the latest generation of LLMs, such as the anticipated GPT-5, to continuously benchmark their capabilities and suitability for educational applications. Future research directions should also include the integration of RAG systems into real classroom activities, as well as measuring their impacts on learning practices and outcomes through long-term research. Lu et al. showed that instruction supported by generative AI considerably improved teachers' self-efficacy and high-level cognitive skills, and therefore RAG systems could simultaneously promote students' learning and teachers' improvement [21]. Based on our findings, we suggest the following applications as promising research avenues for practical school implementations:

- 1) Debugging Assistance Systems: A RAG system enhanced with CoT prompting can assist students in identifying and resolving programming errors.
- 2) Conceptual Knowledge Reinforcement: For theoretical topics in computer science, including computational complexity, and data structures, TSB-style prompting combined with re-ranking retrieval supports the composition of structured descriptions by students.

In addition, there are also further opportunities for integration with other education technologies. For example, integration of RAG systems with automated grading tools has the potential to provide more comprehensive education assistance. The other direction includes the design of models for teacher-AI collaboration for the examination of how RAG systems can effectively support classroom teaching [22]. The objective of the future investigations is ensuring that AI tools serve true education purposes and make authentic contributions to both teaching and learning, rather than operating independently as tools.

VI. CONCLUSION

This study systematically evaluated how model type, prompt engineering, and retrieval design impact RAG response quality in programming education. Our key findings reveal that structured reasoning prompts (e.g., Chain-of-Thought) significantly narrow the performance gap between commercial and open-source models, while the re-ranking strategy consistently proves to be the most effective method for improving retrieval and generation quality.

The main contribution of this study is providing empirical guidance for designing cost-effective, high-quality AI-assisted educational systems. We demonstrate that optimized open-source models can offer a viable alternative to costly commercial solutions, helping educational institutions balance performance with implementation costs. For educational institutions with limited resources, our findings suggest that a lightweight open-source model like Ministral-8b, when combined with a Re-ranking retrieval strategy and

Chain-of-Thought (CoT) prompting, offers a cost-effective solution with comparable performance in response relevance. For applications where factual accuracy is paramount, commercial models like Claude-3.7-Sonnet demonstrated superior faithfulness. Future research should extend these findings to real-world classroom assessments and the development of domain-specific RAG optimizations.

Ultimately, this study confirms the practical value of RAG in creating more personalized and accessible learning experiences in programming education. Specifically, to translate our findings into practice, we envision two ready-to-deploy classroom utilities: (i) an LMS-integrated debugging assistant that delivers lecture-anchored hints and stepwise rationales, and (ii) a concept-reinforcement agent that assembles concise, citation-linked explanations from approved course materials. We plan to evaluate these deployments in semester-length studies using class- or section-level randomization, pre-/post-learning measures aligned with course outcomes, rubric-based blind grading of open responses, and log-based engagement and time-on-task analysis.

APPENDIX PROMPT TEMPLATES AND BUDGETS

General rules. Use only the retrieved context; if context is insufficient, answer “I don’t know.” Avoid external knowledge or uncited claims.

Baseline

System: Answer strictly from context; keep answers concise (≤ 150 words).

User: *Question:* {Q}; *Context:* {top-k passages}.

Budget: Instruction ≤ 120 tokens; total prompt typically ≤ 1500 tokens.

Chain-of-Thought (CoT)

System: Reason step-by-step (Plan $\rightarrow$ Steps $\rightarrow$ Final answer).

User: *Question:* {Q}; *Context:* {top-k passages}.

Budget: Lead-in ≤ 150 tokens; total prompt typically ≤ 1700 tokens.

Few-shot (k = 3)

System: Follow the examples; ground in context.

User: 3 examples ({EXi_Q} $\rightarrow$ {EXi_A ≤ 120 tokens}), then *Question:* {Q}; *Context:* {top-k passages}.

Budget: Examples ≤ 360 tokens; total prompt typically ≤ 1800 tokens.

Rephrase-and-Respond (RaR)

System: First restate the question, then answer using context.

User: Step 1: “The question asks: {restatement}.”
Step 2: {answer grounded in context}.

Budget: Lead-in ≤ 120 tokens; total prompt typically ≤ 1600 tokens.

Take-a-Step-Back (TSB)

System: Abstract key concepts, then give the final answer.

User: *Concepts/Principles:* {2–4 bullets from context}; *Final answer:* {answer}.

Budget: Abstraction ≤ 180 tokens; total prompt typically ≤ 1600 tokens.

Note. These verbatim templates match the strategies in Methods; Few-shot uses $k = 3$.

ACKNOWLEDGMENT

During the writing process, the authors used generative artificial intelligence tools to assist with English translation and sentence refinement, and to help organize and synthesize relevant literature. All content was reviewed and edited by them to ensure accuracy and originality.

REFERENCES

- [1] Z. Ahmed, S. S. Shanto, and A. I. Jony, “Potentiality of generative AI tools in higher education: Evaluating ChatGPT’s viability as a teaching assistant for introductory programming courses,” *STEM Educ.*, vol. 4, no. 3, pp. 165–182, 2024.
- [2] R. Yilmaz and F. G. K. Yilmaz, “The effect of generative artificial intelligence (AI)-based tool use on students’ computational thinking skills, programming self-efficacy and motivation,” *Comput. Educ., Artif. Intell.*, vol. 4, Jan. 2023, Art. no. 100147.
- [3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 9459–9474.
- [4] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” *ACM Comput. Surv.*, vol. 57, no. 5, pp. 1–45, May 2023.
- [5] Y. Pan and R. H. Nehm, “Large language model and traditional machine learning scoring of evolutionary explanations: Benefits and drawbacks,” *Educ. Sci.*, vol. 15, no. 6, p. 676, May 2025.
- [6] S. Jacobs and S. Jäschke, “Leveraging lecture content for improved feedback: Explorations with GPT-4 and retrieval augmented generation,” in *Proc. 36th Int. Conf. Softw. Eng. Educ. Training*, Jul. 2024, pp. 1–5.
- [7] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “Ragas: Automated evaluation of retrieval augmented generation,” in *Proc. Conf. Empirical Methods Natural Lang. Process., Ind. Track (EMNLP)*, Dec. 2023, pp. 460–470.
- [8] S. Berrezueta-Guzman, S. Krusche, and S. Wagner, “From coders to critics: Empowering students through peer assessment in the age of AI copilots,” in *Proc. Int. Symp. Educ. Technol. (ISET)*, Jul. 2025, pp. 66–71.
- [9] P. Haindl and G. Weinberger, “Does ChatGPT help novice programmers write better code? Results from static code analysis,” *IEEE Access*, vol. 12, pp. 114146–114156, 2024.
- [10] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang, “Retrieval augmented language model pre-training,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, Jul. 2021, pp. 3110–3121.
- [11] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, “Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing,” *ACM Comput. Surveys*, vol. 55, no. 9, pp. 1–35, Sep. 2023.
- [12] T. B. Brown et al., “Language models are few-shot learners,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2022, pp. 1877–1901.
- [13] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” *Adv. Neural Inf. Process. Syst.*, 2022, pp. 24824–24837.
- [14] Y. Deng, W. Zhang, Z. Chen, and Q. Gu, “Rephrase and respond: Let large language models ask better questions for themselves,” 2023, *arXiv:2311.04205*.
- [15] H. S. Zheng, S. Mishra, X. Chen, H.-T. Cheng, E. H. Chi, Q. V. Le, and D. Zhou, “Take a step back: Evoking reasoning via abstraction in large language models,” 2023, *arXiv:2310.06117*.
- [16] M. Glass, G. Rossiello, M. F. M. Chowdhury, A. Naik, P. Cai, and A. Gliozzo, “Re2G: Retrieve, rerank, generate,” in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Human Lang. Technol.*, Dec. 2022, pp. 2701–2715.
- [17] K. Sawarkar, A. Mangal, and S. R. Solanki, “Blended RAG: Improving RAG (Retriever-augmented Generation) accuracy with semantic search and hybrid query-based retrievers,” in *Proc. IEEE 7th Int. Conf. Multimedia Inf. Process. Retr. (MIPR)*, vol. 24, Aug. 2024, pp. 155–161.

- [18] P. Qi, “Answering complex open-domain questions with iterative query generation,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Nov. 2021, pp. 1323–1335.
- [19] R. Nogueira and K. Cho, “Passage re-ranking with BERT,” in *Proc. Adv. Inf. Retr., ECIR*. Cham, Switzerland: Springer, Apr. 2020, pp. 632–646.
- [20] D. Wood, J. S. Bruner, and G. Ross, “The role of tutoring in problem solving,” *J. Child Psychol. Psychiatry*, vol. 17, no. 2, pp. 89–100, Mar. 1976.
- [21] J. Lu, R. Zheng, Z. Gong, and H. Xu, “Supporting teachers’ professional development with generative AI: The effects on higher order thinking and self-efficacy,” *IEEE Trans. Learn. Technol.*, vol. 17, pp. 1267–1277, 2024.
- [22] S.-C. Kong and Y. Yang, “A human-centered learning and teaching framework using generative artificial intelligence for self-regulated learning development through domain knowledge learning in K–12 settings,” *IEEE Trans. Learn. Technol.*, vol. 17, pp. 1562–1573, 2024.

CHRISTOPHER C. Y. YANG received the Ph.D. degree from the Graduate School of Informatics, Kyoto University, Japan, in 2023. He is currently an Assistant Professor with the Department of Computer Science, National Taipei University of Education, Taiwan. His research interests include learning analytics, machine learning, and personalized learning. Specifically, he is an active researcher in applying learning analytics and machine learning methodologies for improving the personalized learning experiences of learners in various education scenarios, such as self-regulated learning, flipped learning, blended learning, and computer-assisted language learning.

GUAN-JIE LIU received the M.S. degree from the Department of Computer Science and Information Engineering, National Central University, Taiwan, in 2025. His primary research interests include artificial intelligence, machine learning, large language models, and retrieval-augmented generation. His recent work centers on leveraging LLM-based systems for knowledge-intensive applications, with an emphasis on enhancing information retrieval, response generation, and task automation using retrieval-augmented techniques.

CHENG-JUI YU received the B.S. degree in computer science information and management from Soochow University. He is currently pursuing the master’s degree with the Department of Computer Science and Information Engineering, National Central University, Taiwan. He developed a strong foundation in programming and data analytics with Soochow University. His research interests include natural language processing (NLP), retrieval-augmented generation (RAG), and generative AI. His work focuses on large-scale data processing, automation pipelines, and multimodal content integration. His current research aims to explore the application of generative AI in personalized learning and intelligent educational systems.

...